Generics: Generics und Polymorphie

Carsten Gips (HSBI)

TL;DR
Auch mit generischen Klassen stehen die Mechanismen Vererbung und Überladen zur Verfügung. Dabei muss aber beachtet werden, dass generische Klassen sich **"invariant"** verhalten: Der Typ selbst folgt der Vererbungsbeziehung, eine Vererbung des Typ-Parameters begründet *keine* Vererbungsbeziehung! D.h. aus `U extends O` folgt **nicht** `A<U> extends A<O>`.
Bei Arrays ist es genau anders herum: Wenn `U extends O` dann gilt auch `U[] extends O[]` … (Dies nennt man "*kovariantes*" Verhalten.)

Videos

- VL Generics und Polymorphie

Unterlagen

- PDF-Version

---

1 Generische Polymorphie

`B<E> extends A<E>`

```
class A<E> { ... }
class B<E> extends A<E> { ... }

A<Double> ad = new B<Double>();
A<String> as = new B<String>();
```

```
class Vector<E> { ... }
class Stack<E> extends Vector<E> { ... }

Vector<Double> vd = new Stack<Double>();
Vector<String> vs = new Stack<String>();
```

=> Polymorphie bei Generics bezieht sich auf **Typ** (nicht Typ-Parameter)

Invarianz: Generics sind *invariant*, d.h. ein `HashSet<String>` ist ein Untertyp von `Set<String>`. Bei der Vererbung muss der Typ-Parameter identisch sein.

2 Polymorphie bei Generics bezieht sich nur auf Typ!

"`B extends A`" **bedeutet nicht** "`C<B> extends C<A>`"

```java
Stack<Number> s = new Stack<Integer>(); // DAS GEHT SO NICHT!

// Folgen (wenn obiges gehen wuerde):
s.push(new Integer(3)); // das ginge sowieso ...

// Folgen (wenn obiges gehen wuerde):
// Stack<Number> waere Oberklasse auch von Stack<Double>
s.push(new Double(2.0)); // waere dann auch erlaubt ...

// Das Objekt (Stack<Integer>) kann aber keine Double speichern!
// Zur Laufzeit keine Typ-Informationen mehr!
```

- Typ-Löschung => zur Laufzeit keine Typinformationen vorhanden
- Compiler muss Typen prüfen (können)!

3 Abgrenzung: Polymorphie bei Arrays

Wenn "`B extends A`" dann "`B[] extends A[]`"

```java
Object[] x = new String[] {"Hello", "World", ":-)"};
x[0] = "Hallo";
x[0] = new Double(2.0);  // Laufzeitfehler
String[] y = x;  // String[] ist KEIN Object[]!!!
```

- Arrays besitzen Typinformationen über gespeicherte Elemente
- Prüfung auf Typ-Kompatibilität zur **Laufzeit** (nicht Kompilierzeit!)

Hinweis auf Java-Geschichte (Java-Insel: "Type Erasure")

Arrays gab es sehr früh, Generics erst relativ spät (ab Java6) => bei Arrays fand man das Verhalten natürlich und pragmatisch (trotz der Laufzeit-Überprüfung).

Bei der Einführung von Generics musste man Kompatibilität sicherstellen (alter Code soll auch mit neuen Compilern übersetzt werden können - obwohl im alten Code Raw-Types verwendet werden). Außerdem wollte man von Laufzeit-Prüfung hin zu Compiler-Prüfung. Da würde das von Arrays bekannte Verhalten Probleme machen ...

Kovarianz: Arrays sind *kovariant*, d.h. ein Array vom Typ `String[]` ist wegen `String extends Object` ein Untertyp von `Object[]`.

Beispiel arrays.X

4 Arrays vs. parametrisierte Klassen

=> Keine Arrays mit parametrisierten Klassen!

```
Foo<String>[] x = new Foo<String>[2];   // Compilerfehler

Foo<String[]> y = new Foo<String[]>();  // OK :)
```

Arrays mit parametrisierten Klassen sind nicht erlaubt! Arrays brauchen zur Laufzeit Typinformationen, die aber durch die Typ-Löschung entfernt werden.

5 Diskussion Vererbung vs. Generics

Vererbung:

- *IS-A*-Beziehung
- Anwendung: Vererbungsbeziehung vorliegend, Eigenschaften verfeinern
- Beispiel: Ein Student *ist eine* Person

Generics:

- Schablone (Template) für viele Datentypen
- Anwendung: Identischer Code für unterschiedliche Typen
- Beispiel: Datenstrukturen, Algorithmen generisch realisieren

6 Wrap-Up

- Generics: Vererbung und Überladen möglich, aber:
  Aus "`U extends O`" folgt nicht** "`A<U> extends A<O>`"**

- Achtung: Bei Arrays gilt aber: Wenn "`U extends O`" dann gilt auch "`U[] extends O[]`" ...

7 Zum Nachlesen

- Ullenboom (2021, Kap. 11.5)
- Oracle Corporation (2025)
- Oracle Corporation (2024)
- Bloch (2018)

Lernziele

- k3: Ich kann Vererbungsbeziehungen mit generischen Klassen bilden
- k3: Ich kann mit Arrays und generischen Typen umgehen

Quellen

Bloch, J. 2018. *Effective Java*. 3. Aufl. Addison-Wesley.
Oracle Corporation. 2024. „The Java Tutorials“. 2024. https://docs.oracle.com/javase/tutorial/.
———. 2025. „Learn Java“. 2025. https://dev.java/learn/.
Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 89adc60 (lecture: improve learning outcomes (Generics Polymorphism), 2025-05-26)